

Unit 3 – Amortized Analysis

CS 201 - Data Structures II

Spring 2024

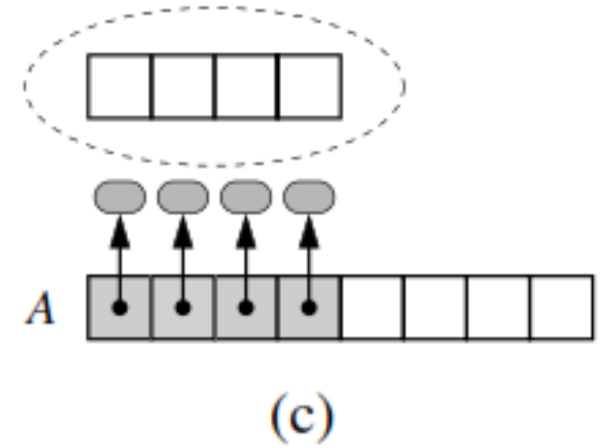
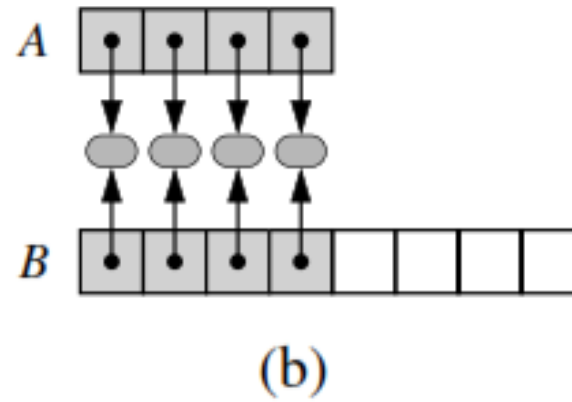
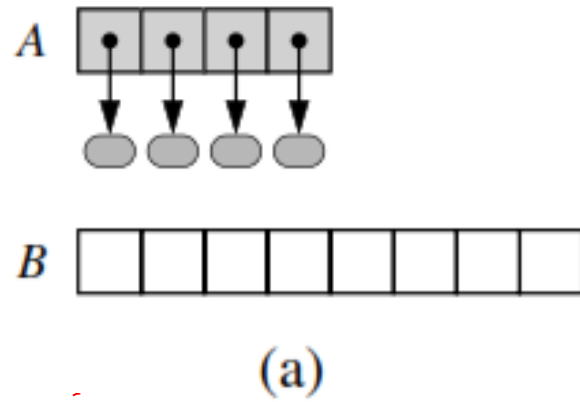
Habib University

Syeda Saleha Raza

Outline

- Amortized Analysis
 - Resize() method
 - Intuition
 - Aggregate Method
 - Accounting Method

Resizing an array



Amortized Analysis

- Amortized Analysis is used for algorithms where an occasional operation is very slow, but most of the other operations are faster.
- if a data structure has an amortized running time of $f(n)$, then a sequence of m operations takes at most $mf(n)$ time. Some individual operations may take more than $f(n)$ time but the average, over the entire sequence of operations, is at most $f(n)$.

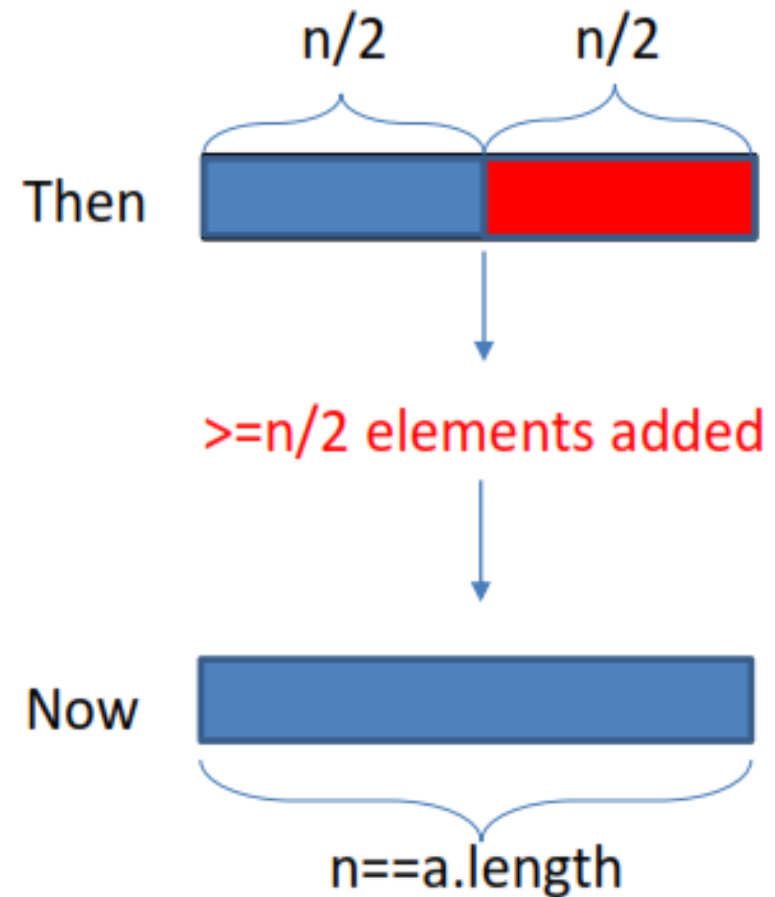
Proposition 5.1: *Let S be a sequence implemented by means of a dynamic array with initial capacity one, using the strategy of doubling the array size when full. The total time to perform a series of n append operations in S , starting from S being empty, is $O(n)$.*

Resize() - Analysis

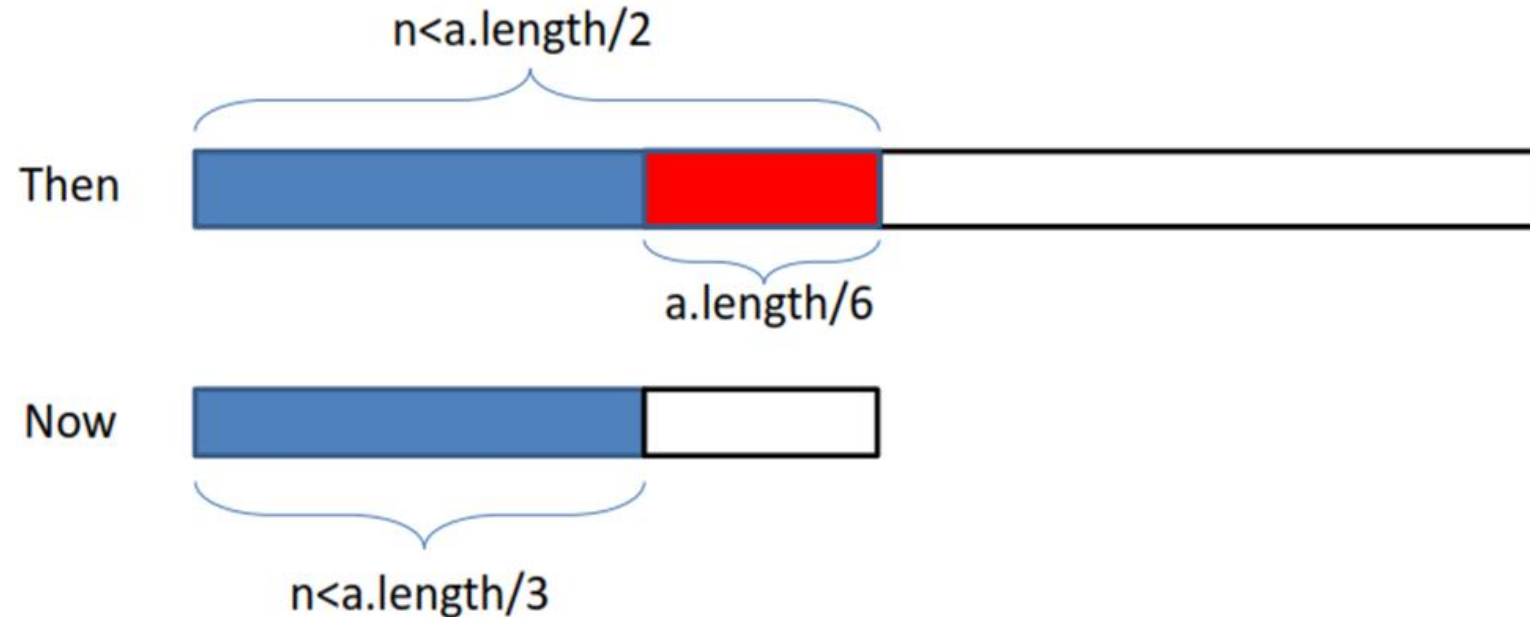
- How many elements would at least be added between two calls to `resize()`?

Resize() – Number of add() calls

If a resize operation copies n elements, then there have been at least $n/2$ add or remove operations since the preceding resize operation.



Resize() – Number of remove() calls



- If a resize operation copies n elements, then there have been at least $n/2$ ($(a.length/3)/2$) add or remove operations since the preceding resize operation.

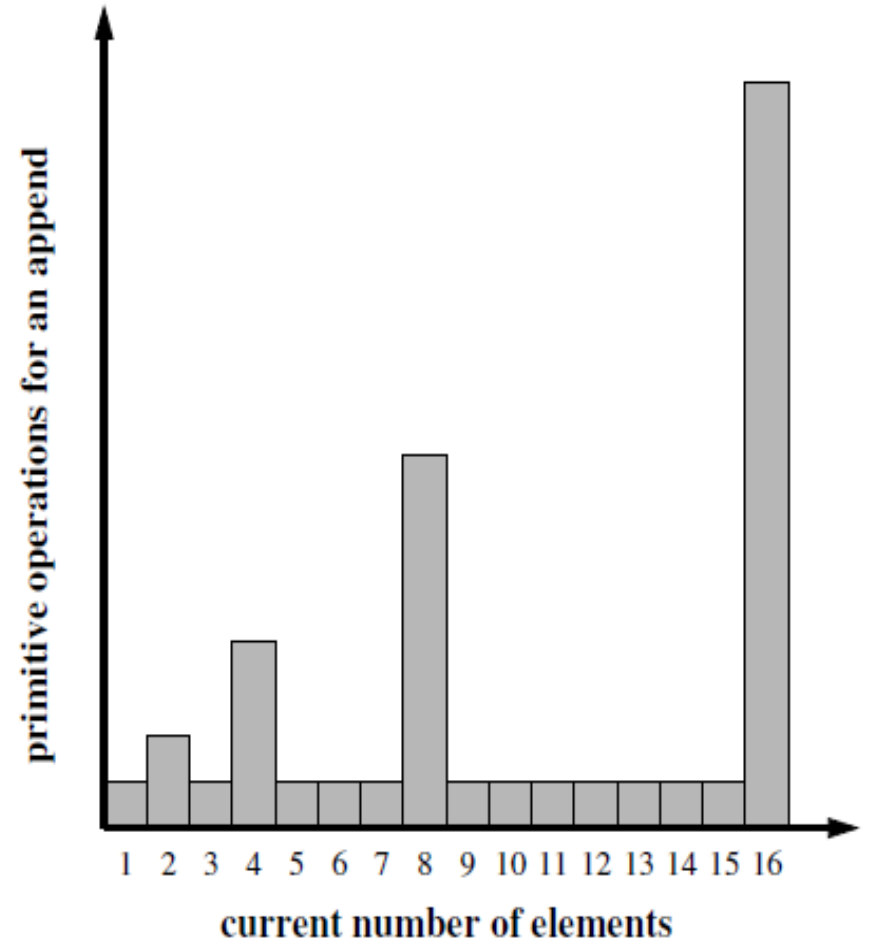
Aggregate Method

- Determine the worst-case cost of the entire sequence of operations, $T(n)$.
- Divide this cost by the number of operations in the sequence, n .

Aggregate Method – Dynamic Array

$T(n)$ = Cost of all add/remove calls +
Cost of all `resize()` calls

Let's calculate this cost and take its average over n number of add/remove operations.



Aggregate Method – Dynamic Array

$$\begin{aligned}\sum_{j=0}^{\log(n-1)} 2^j &= 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{\log_2(n-1)} \\ &= a_0(r^m - 1)/(r - 1) \text{ (sum of geometric series formula)}\end{aligned}$$

First term: $a_0 = 2^0 = 1$

Ratio between terms: $r = 2$

Total terms: $m = \log_2(n-1) + 1$

$$\begin{aligned}&= (2^{\log_2(n-1)+1} - 1)/(2 - 1) \\ &= (2 \cdot 2^{\log_2(n-1)} - 1) = (2(n-1) - 1) = 2n - 2 - 1 = 2n - 3 \\ \sum_{j=0}^{\log(n-1)} 2^j &= 2n - 3\end{aligned}$$

- Amortized cost of n add operations $T(n) = n + 2n - 3 = 3n - 3 < 3n = O(n)$
- Amortized cost of 1 add operation $= T(n)/n = 3n/n = 3 = O(1)$

Accounting Method

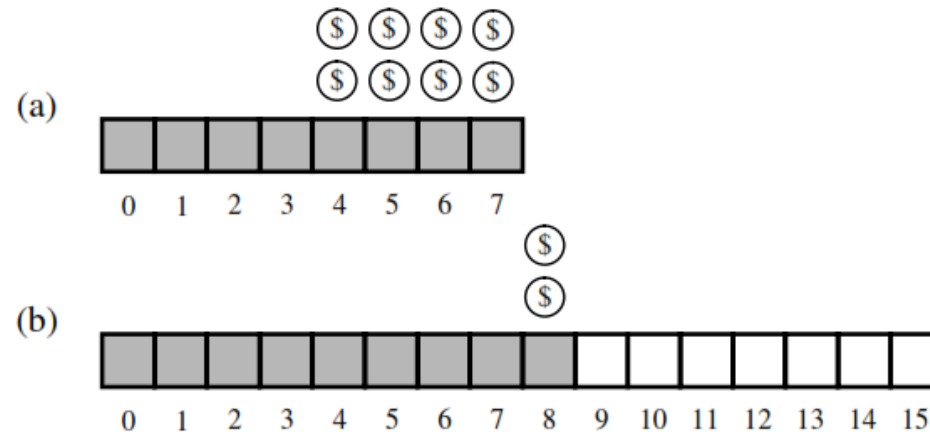
Accounting

- Accounting method involves charging the cost of some steps to some other, earlier steps. The method is similar to taxation, except that we focus on where each unit of tax is (or will be) spent, rather than where is it collected.
- By charging the cost of some operations to earlier operations, we are overestimating the total cost of any sequence of operations, since we pay for some charges from future operations that may never actually occur.

Accounting Method

Each time an m^{th} element is inserted, 3 units (instead of 1) are charged:

- One unit is used to insert *that element* immediately into the table.
- One unit is used to move *that element* the first time the table is grown afterwards m is inserted.
- One unit is donated to the corresponding element in the first half of the array, and is used to move that element the first time the table is grown after m is inserted.



Accounting Method – resize

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	...
ni																				
ci																				
Credited																				
Total Credit																				

- Cost of i^{th} add (C_i) =
$$\begin{cases} i, & \text{when } (i-1) \text{ is a power of } 2 \\ 1, & \text{otherwise} \end{cases}$$

- Cost of n add operations = $n + \sum_{j=0}^{\log_2(n-1)} 2^j$

Accounting Method – resize

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	...
ni	1	2	4	4	8	8	8	8	16	16	16	16	16	16	16	16	32	32	32	...
ci	1	2	(1+2)	1	(1+4)	1	1	1	(1+8)	1	1	1	1	1	1	1	(1+16)	1	1	...
Total Credit		2	2	4	2	4	6	8	2	4	6	8	10	12	14	16	2	4	6	

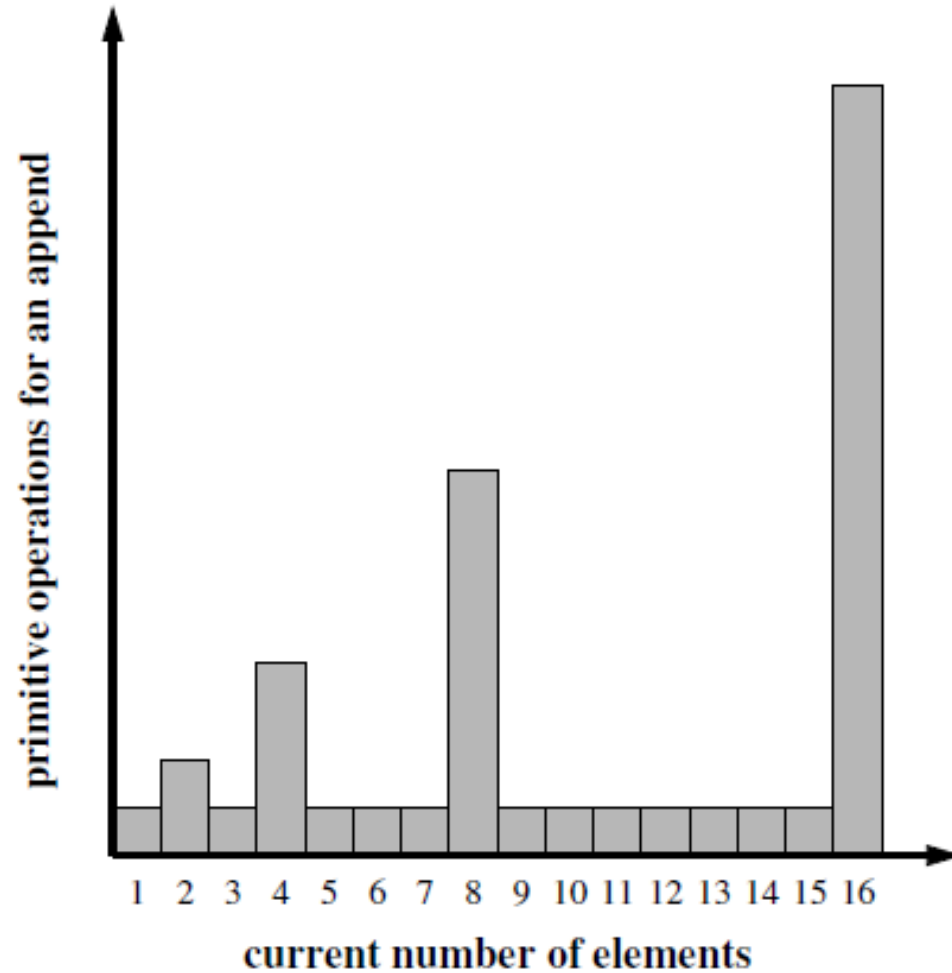
- Cost of i^{th} add (C_i) = $\begin{cases} i, & \text{when } (i-1) \text{ is a power of 2} \\ 1, & \text{otherwise} \end{cases}$

- Cost of n add operations = $n + \sum_{j=0}^{\log_2(n-1)} 2^j$

Amortized Analysis

- Increasing the size by:
 - A constant factor
 - A constant number of cells

Increasing by a constant factor



Discussion

- What would be the complexity if, instead of doubling the array on `resize()`, you increase its size by a quarter of it?

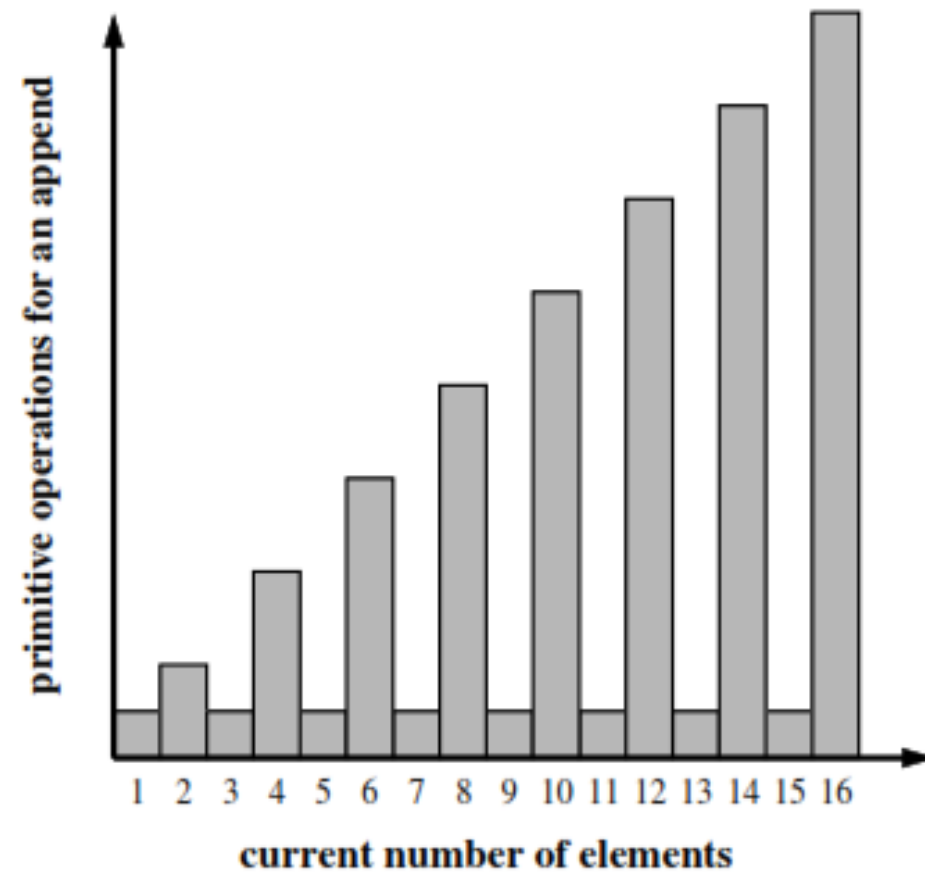
Discussion

- What if you increase its size by a constant number of elements, say 10?

Beware of arithmetic progression

Proposition 5.2: *Performing a series of n append operations on an initially empty dynamic array using a fixed increment with each resize takes $\Omega(n^2)$ time.*

Increasing by a constant number



(a)

Increasing by a constant number

- Growing the array by 5 elements as it gets full:
 - The number of elements copied during resize would become:
 - $5 + 10 + 15 + 20 + \dots + n$
 - $5(1 + 2 + 3 + 4 + \dots + n/5)$
 - $O(n^2)$
- Growing the array by 50 elements as it gets full
 - The number of copy operations would become:
 - $10 + 20 + 30 + 40 + 50 + \dots + n$
 - $10(1 + 2 + 3 + 4 + \dots + n/10)$
 - $O(n^2)$

Increasing by a constant number

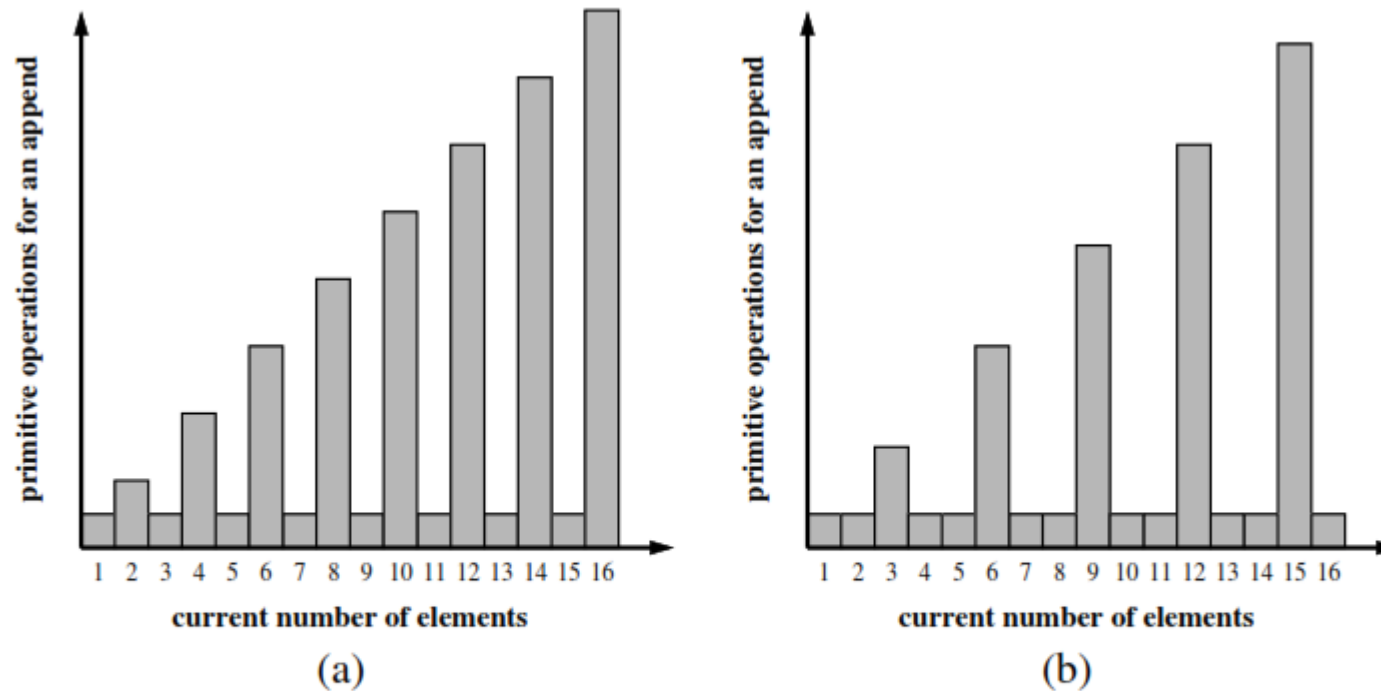


Figure 5.15: Running times of a series of append operations on a dynamic array using arithmetic progression of sizes. (a) Assumes increase of 2 in size of the array, while (b) assumes increase of 3.

Exercise – Amortized analysis of Queue using two stacks

We know that a queue (FIFO) can be implemented using two stacks (LIFO). Say that our stack has three operations, push, pop and isEmpty, each with cost 1. The queue can be implemented as follows:

- enqueue(x): push x onto stack1
- dequeue(): if stack2 is empty then pop the entire contents of stack1 pushing each element in turn onto stack 2. Now pop from stack2 and return the result.
- Worst Case: ?
- Let's analyse its amortized complexity.

Exercise – Amortized analysis of Queue using two stacks

- Considering that,
 - Each elements is pushed at most two times.
 - Each elements is popped at most two times.
- Applying accounting method, what cost should each enqueue() operation be charged to pay for subsequent push()/pop() operations on dequeue().

Resources

- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- [An Introduction to Amortized Analysis | Baeldung on Computer Science](#)

Thanks